A* Motion Planning

In [80]:
```
# The autoreload extension will automatically load in new code as you edit f
# so you don't need to restart the kernel every time
%load_ext autoreload
%autoreload 2
import numpy as np
import matplotlib.pyplot as plt
import scipy.interpolate
from P1_astar import DetOccupancyGrid2D, AStar
from utils import generate_planning_problem
```

```
The autoreload extension is already loaded. To reload it, use:
  %reload_ext autoreload
```

In [81]:
```
# from asl_tb3_lib.navigation import TrajectoryPlan
from utils import TrajectoryPlan
```

Sanity checks

Quick checks of `distance`, `heuristic`, and `get_neighbors` (implement `is_free` and `heuristic` as well before running this cell, since the `AStar` constructor calls `heuristic`). These are not exhaustive, but every assertion should pass before you move on.

In [82]:
```
_empty = DetOccupancyGrid2D(10, 10, [])
_a = AStar((0, 0), (10, 10), (1, 1), (4, 5), _empty)
assert np.isclose(_a.distance((1, 1), (4, 5)), 5.0)

for _h, _expected in [("l1", 7.0), ("l2", 5.0), ("linf", 4.0)]:
    _a = AStar((0, 0), (10, 10), (1, 1), (4, 5), _empty, heuristic=_h)
    assert np.isclose(_a.heuristic((1, 1)), _expected), _h
    assert np.isclose(_a.distance((1, 1), (4, 5)), 5.0), "distance must not

_a = AStar((0, 0), (10, 10), (1, 1), (4, 5), _empty, heuristic="l2", heurist
assert np.isclose(_a.heuristic((1, 1)), 10.0)

assert len(_a.get_neighbors((5, 5))) == 8
assert len(_a.get_neighbors((0, 0))) == 3
print("All sanity checks passed!")
```

```
All sanity checks passed!
```

Simple Environment

(Please submit resulting plot from this section in your write-up)

Workspace

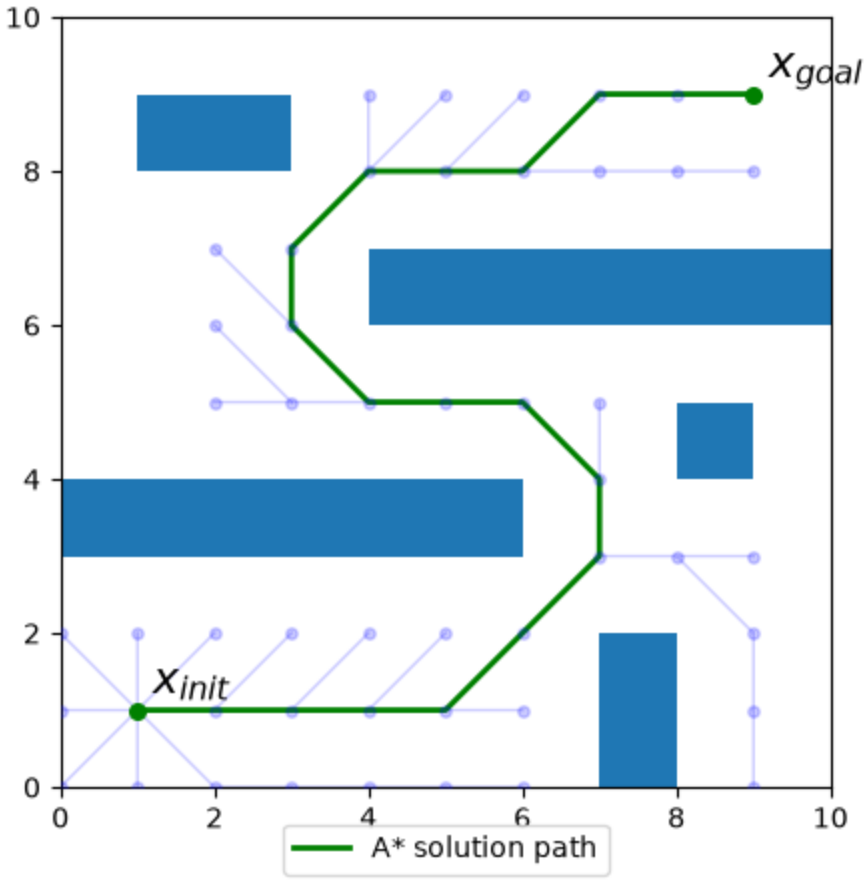
In [83]:
```
width = 10
height = 10
obstacles = [((0,3),(6,4)),((4,6),(10,7)),((7,0),(8,2)),((1,8),(3,9)),((8,4)
occupancy = DetOccupancyGrid2D(width, height, obstacles)
```

Starting and final positions

In [84]:
```
x_init = (1, 1)
x_goal = (9, 9)
```

Run A* planning

In [85]:
```
astar = AStar((0, 0), (width, height), x_init, x_goal, occupancy)
if not astar.solve():
    print("No path found")
else:
    plt.rcParams['figure.figsize'] = [5, 5]
    astar.plot_path()
    astar.plot_tree()
```



Random Cluttered Environment

Generate workspace, start and goal positions

(Try changing these and see what happens, but reset `SEED = 274` and the default parameters before generating the plots for your write-up.)

In [86]:
```
SEED = 274  # use 274 for the plots in your write-up
np.random.seed(SEED)

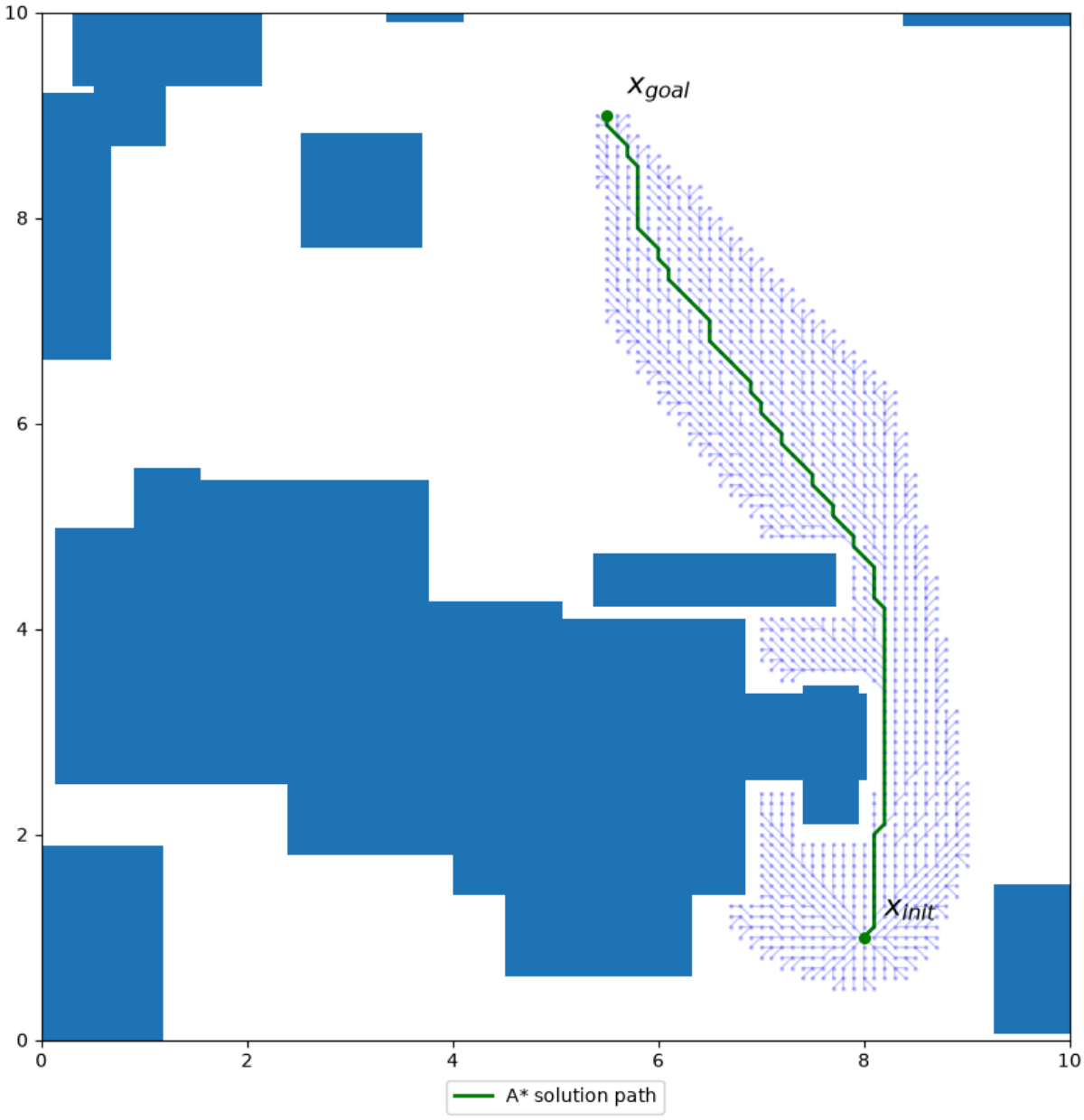
width = 10
height = 10
num_obs = 25
min_size = .5
max_size = 3

occupancy, x_init, x_goal = generate_planning_problem(width, height, num_obs
```

Run A* planning

In [87]:
```
astar = AStar((0, 0), (width, height), x_init, x_goal, occupancy, resolutior
if not astar.solve():
    print("No path found! (This is normal, try re-running the block above)")
else:
```

```
    plt.rcParams['figure.figsize'] = [10, 10]
    astar.plot_path()
    astar.plot_tree(point_size=2)
```



Smooth Trajectory

In [88]:
```
def path_length(path):
    path = np.asarray(path)
    return np.sum(np.linalg.norm(np.diff(path, axis=0), axis=1))

def compute_smooth_plan(path, v_desired=0.15, spline_alpha=0.05) -> Trajectc
    # Ensure path is a numpy array
    path = np.asarray(path)

    # Compute and set the following variables:
    #   1. ts:
```

```
    #      Compute an array of time stamps for each planned waypoint assumin
    #      velocity between waypoints.
    #
    #   2. path_x_spline, path_y_spline:
    #      Fit cubic splines to the x and y coordinates of the path separate
    #      with respect to the computed time stamp array.
    #      Hint: Use scipy.interpolate.splrep
    ##### YOUR CODE STARTS HERE #####

    dist = path_length(path)
    tspan = dist / v_desired
    n_steps = path.shape[0]
    # t = d/v
    ts = np.linspace(0, tspan, n_steps)

    path_x_spline = scipy.interpolate.splrep(ts, path[:,0], k = 3, s = splir
    path_y_spline = scipy.interpolate.splrep(ts, path[:,1], k = 3, s = splir
    ###### YOUR CODE END HERE ######

    return TrajectoryPlan(
        path=path,
        path_x_spline=path_x_spline,
        path_y_spline=path_y_spline,
        duration=ts[-1],
    )
```

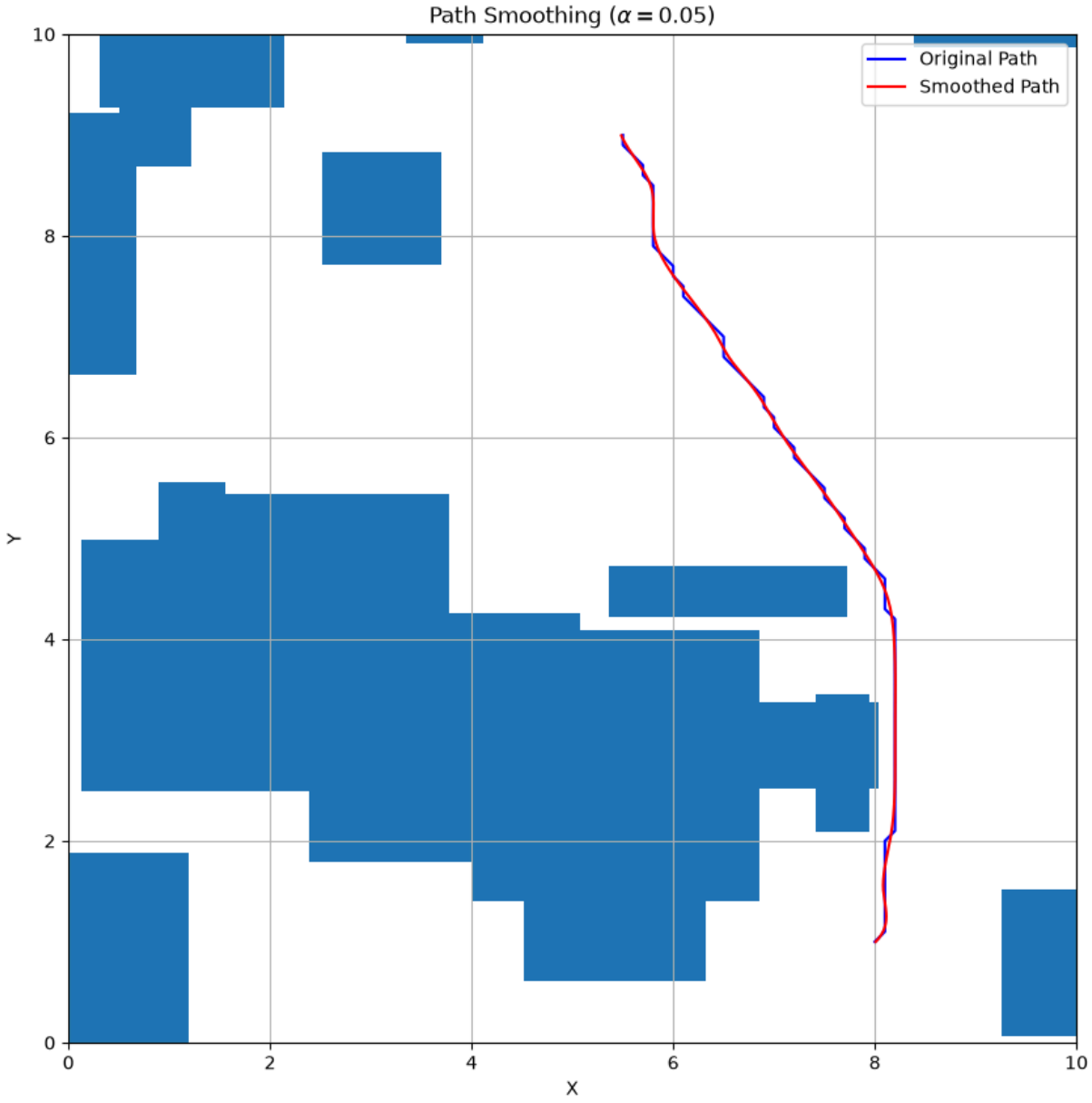
In [89]:

```
# construct a trajectory plan (0.05)
plan = compute_smooth_plan(astar.path, spline_alpha=0.05)

# plot AStar path v.s. smoothed path
astar_path = np.asarray(astar.path)
smoothed_path = plan.smoothed_path()

astar.occupancy.plot() # Added

plt.plot(astar_path[:,0], astar_path[:,1], 'b-', label='Original Path')
plt.plot(smoothed_path[:, 0], smoothed_path[:, 1], 'r-', label='Smoothed Pat
plt.legend()
plt.xlabel('X')
plt.ylabel('Y')
plt.xlim([0, width])
plt.ylim([0, height])
plt.title(r'Path Smoothing ($\alpha = 0.05$)')
plt.grid()
```

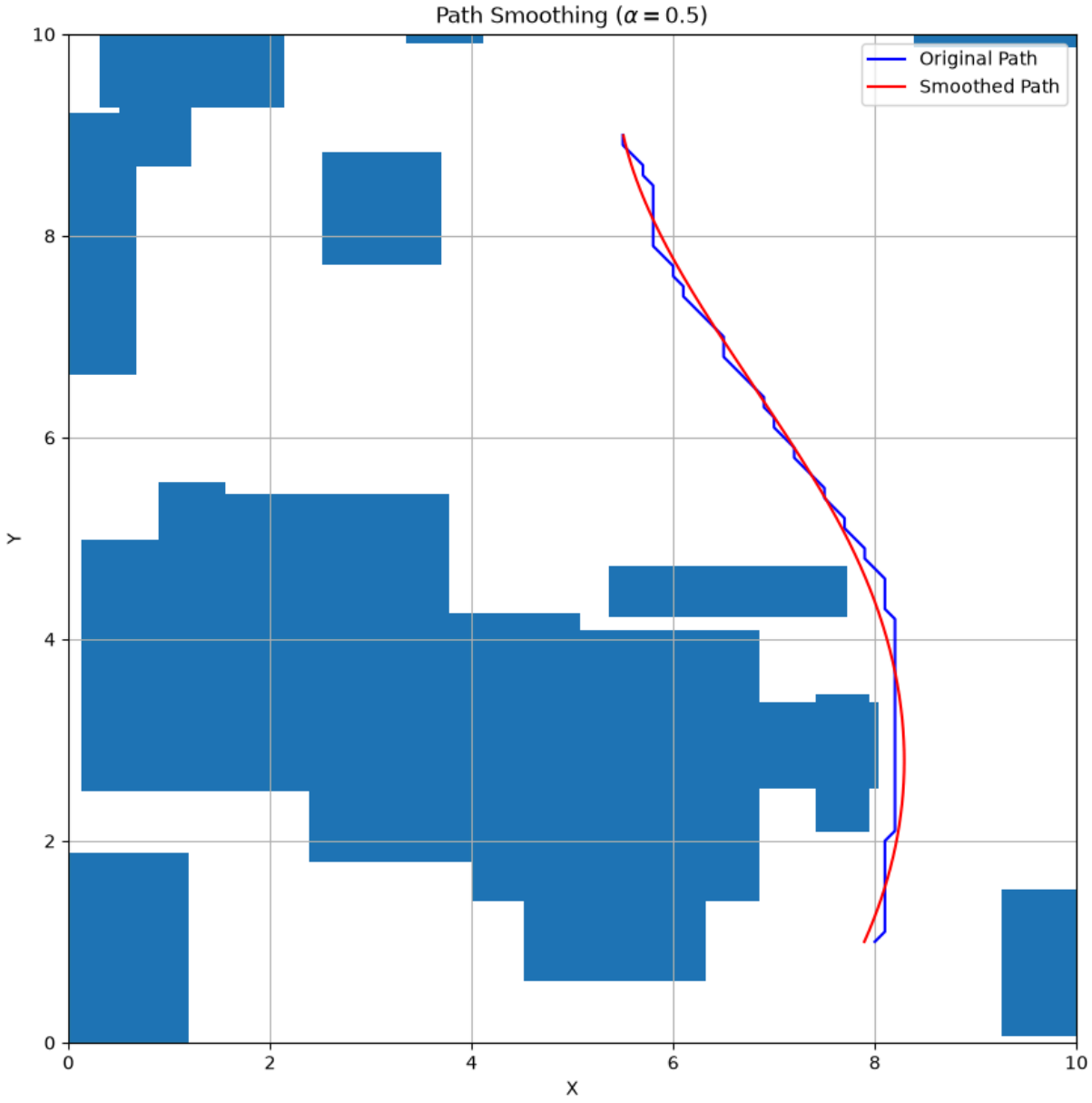


In [90]:
```
# construct a trajectory plan (0.5)
plan = compute_smooth_plan(astar.path, spline_alpha=0.5)

# plot AStar path v.s. smoothed path
astar_path = np.asarray(astar.path)
smoothed_path = plan.smoothed_path()

astar.occupancy.plot() # Added

plt.plot(astar_path[:,0], astar_path[:,1], 'b-', label='Original Path')
plt.plot(smoothed_path[:, 0], smoothed_path[:, 1], 'r-', label='Smoothed Pat
plt.legend()
plt.xlabel('X')
plt.ylabel('Y')
plt.xlim([0, width])
plt.ylim([0, height])
plt.title(r'Path Smoothing ($\alpha = 0.5$)')
plt.grid()
```

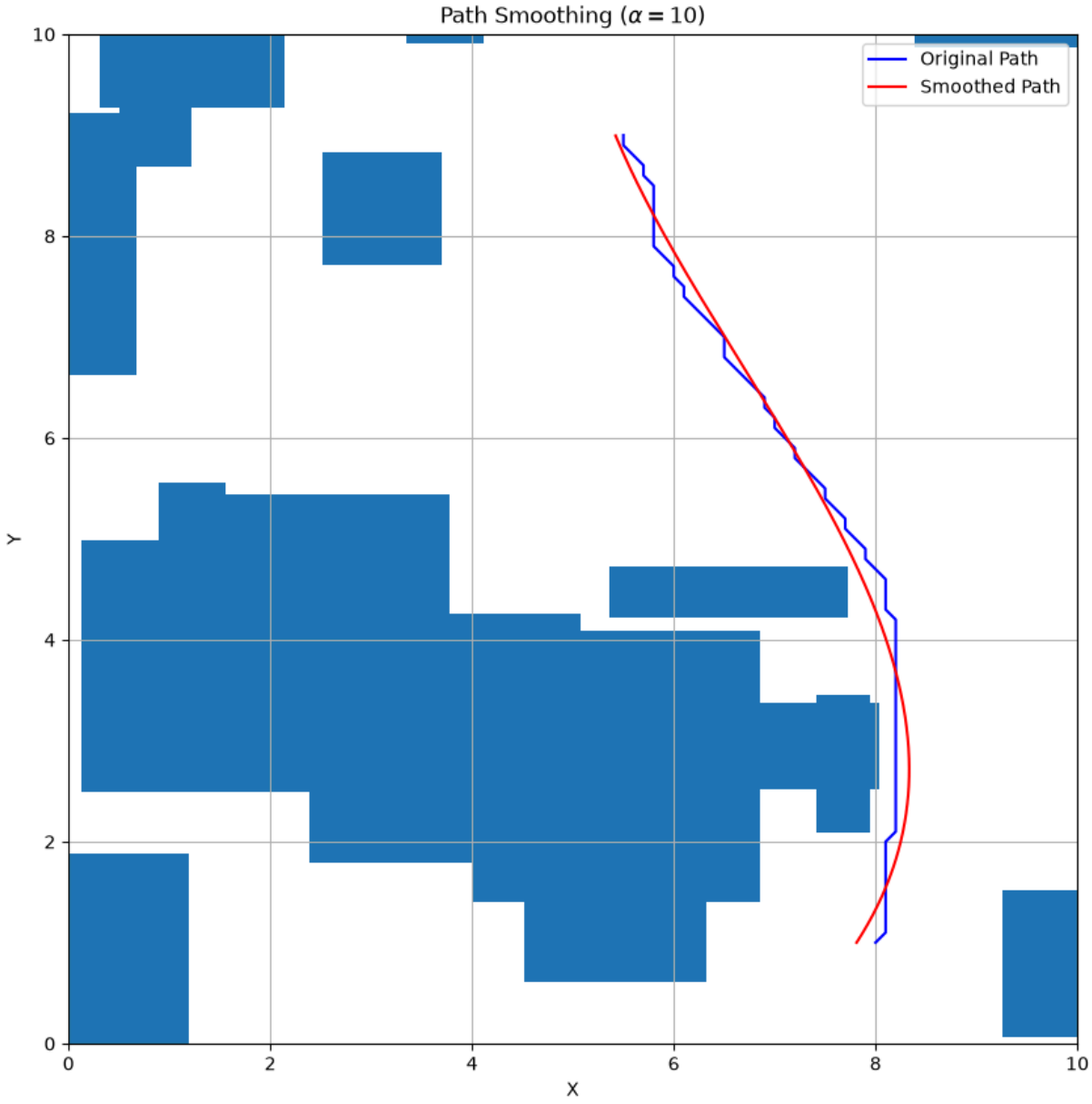


In [91]:
```
# construct a trajectory plan (10)
plan = compute_smooth_plan(astar.path, spline_alpha=10)

# plot AStar path v.s. smoothed path
astar_path = np.asarray(astar.path)
smoothed_path = plan.smoothed_path()

astar.occupancy.plot() # Added

plt.plot(astar_path[:,0], astar_path[:,1], 'b-', label='Original Path')
plt.plot(smoothed_path[:, 0], smoothed_path[:, 1], 'r-', label='Smoothed Pat
plt.legend()
plt.xlabel('X')
plt.ylabel('Y')
plt.xlim([0, width])
plt.ylim([0, height])
plt.title(r'Path Smoothing ($\alpha = 10$)')
plt.grid()
```



Heuristic Comparison

(Please submit the plot and the printed table from this section in your write-up.)

This runs A* on the random environment above (`SEED = 274`) with different heuristic norms and weights. The edge cost is always the Euclidean `distance` .

In [96]:
```
configs = [("l2", 1.0), ("l1", 1.0), ("linf", 1.0), ("l2", 3.0)]
colors = ["green", "red", "orange", "purple"]

plt.rcParams['figure.figsize'] = [10, 10]
occupancy.plot()
print(f"{'heuristic':>9} | {'w':>4} | {'path length':>11} | {'nodes expandec
print("-" * 49)
for (heuristic, weight), color in zip(configs, colors):
    astar_h = AStar((0, 0), (width, height), x_init, x_goal, occupancy, resc
```

```
                    heuristic=heuristic, heuristic_weight=weight)
    if not astar_h.solve():
        print(f"{heuristic:>9} | {weight:>4.1f} | no path found")
        continue
    print(f"{heuristic:>9} | {weight:>4.1f} | {path_length(astar_h.path):>11
    p = np.asarray(astar_h.path)
    plt.plot(p[:, 0], p[:, 1], color=color, linewidth=2, label=f"{heuristic}
plt.legend()
plt.title("A* paths for different heuristics")
plt.show()
```

```
heuristic |    w | path length | nodes expanded
-----------------------------------------------
       l2 |  1.0 |       9.201 |           1091 | 9.201219330881964
       l1 |  1.0 |       9.201 |            318 | 9.201219330881973
     linf |  1.0 |       9.201 |           1945 | 9.201219330881965
       l2 |  3.0 |       9.898 |            146 | 9.898275605729687
```

